Лабораторна робота №7.

Тема: Функціональна обробка масивів: фільтрація, трансформація та агрегація даних.

Мета: Навчитися створювати шаблони пошуку, перевіряти коректність введення даних та вилучати підрядки з тексту.

КОРОТКІ ТЕОРЕТИЧНІ ВІДОМОСТІ

Сучасна обробка масивів базується на принципі незмінності (immutability): замість модифікації існуючого масиву ми створюємо новий на основі певних правил.

1. Фільтрація (`filter`)

Метод створює новий масив з усіма елементами, які пройшли перевірку, задану у функції зворотного виклику (callback).

Результат: Масив тієї ж довжини або менший.

Callback: Має повертати `true` (залишити) або `false` (видалити).

Синтаксис:

```
const newArray = originalArray.filter(callbackFunction)
```

Як це працює

- **Ітерація:** Метод`filter()` виконує ітерацію по кожному елементу масиву.
- **Перевірка умови:** Функція зворотного виклику викликається для кожного елемента та повинна повертати `boolean` значення (або `true` / `false`).
- **Створення нового масиву:**
  - Якщо функція зворотного виклику повертає значення `true` для елемента, цей елемент включається до нового масиву.
  - Якщо повертає `false`, елемент пропускається.
- **Незмінність:** оригінальний масив не змінюється; `filter()` завжди повертає поверхневу копію з відфільтрованими результатами

Практична застосування `filter (Відбір)`

Уявіть сито. Ви кажете: «Залишити тільки ті числа, що більші за 10».

Приклад: `[5, 12, 8].filter(x => x > 10)` → результат [12].

2. Трансформація (`map`)

Метод створює новий масив, у якому кожен елемент є результатом перетворення оригінального елемента.

Результат: Масив завжди тієї ж довжини, що й оригінальний.

Callback: Повертає нове значення елемента.

Синтаксис:

```
const newArray = originalArray.map(callbackFunction)
```

Ключова особливість: Розмір нового масиву завжди такий самий, як і у вихідного масиву. Вихідний масив не змінюється.

Практичне застосування `map (Перетворення)`

Уявіть конвеєр. Кожен елемент заходить, з ним щось роблять і він виходить зміненим. Кількість елементів не змінюється.

Приклад: `[1, 2, 3].map(x => x * 2)` → результат [2, 4, 6].

3. Агрегація (`reduce`)

Метод застосовує функцію до акумулятора та кожного значення масиву (зліва направо), щоб звести його до одного значення.

Результат: Одне значення (число, рядок, об'єкт або навіть новий масив).

Аргументи: Акумулятор (результат попередньої ітерації) та поточний елемент.

Практичне застосування `reduce (Стиснення/Агрегація)`

Це як збирати пазл або рахувати гроші в гаманці. Ви берете всі елементи та перетворюєте їх на одне число (суму, добуток тощо).

Приклад: `[1, 2, 3].reduce((sum, x) => sum + x, 0)` → результат 6.

Як це працює

Метод `reduce()` перебирає кожен елемент масиву, виконуючи для кожного елемента надану функцію зворотного виклику "`reducer`"

Синтаксис

```
array.reduce(callbackFunction, initialValue);
```

`callbackFunction` – функція, що виконується для кожного елемента масиву. Вона приймає чотири аргументи:

- `accumulator(або acc)`: Значення, отримане в результаті попереднього виклику функції зворотного виклику. Вона накопичує кінцевий результат.
- `currentValue(або item)`: Поточний елемент масиву, що обробляється.
- `currentIndex` (необов'язково): Індекс поточного елемента.
- `Arrayreduce` (необов'язково): Було викликано оригінальний масив .

`initialValue` (необов'язково): Значення, яке використовуватиметься як перший аргумент першого виклику функції `callbackFunction`. Якщо початкове значення не вказано, перший елемент масиву використовується як початковий акумулятор, а ітерація починається з другого елемента.

Типові випадки використання

- **Підсумовування чисел:** Обчислення загальної суми всіх елементів масиву.
- **Зведення масиву до рівності:** перетворення вкладеної структури масиву в однорівневий масив.
- **Трансформація структур даних:** агрегування даних в один об'єкт або новий масив

4. Пошук та перевірка (find, every, some)

- find: повертає перший елемент, що відповідає умові.
- every: повертає true, якщо всі елементи відповідають умові.
- some: повертає true, якщо хоча б один елемент відповідає умові.

Практичне застосування

Уявімо, що у нас є масив оцінок студента: `const grades = [80, 95, 55, 70, 100];`

1. Метод find (Пошук першого)

Він шукає у масиві елемент, поки не знайде перший, що підходить під умову. Як тільки знайшов –повертає його і зупиняється.

```
// Знайти першу оцінку, яка нижче прохідного бала (60)
const failedGrade = grades.find(x => x < 60);

console.log(failedGrade); // 55
```

2. Метод every (Перевірка всіх)

Він працює як суворий контролер. Повертає `true` тільки якщо абсолютно всі елементи відповідають умові.

```
// Чи всі оцінки є позитивними (більше 0)?
const allPositive = grades.every(x => x > 0);

console.log(allPositive); // true (бо всі числа > 0)

// Чи всі оцінки вищі за 90?
const allHigh = grades.every(x => x > 90);

console.log(allHigh); // false (бо є 80, 55 тощо)
```

3. Метод some (Хоча б один)

Він набагато "лояльніший". Йому достатньо знайти хоча б один елемент, що задовольняє умову, щоб сказати `true`.

```
// Чи є в масиві хоча б одна ідеальна оцінка (100)?
const hasPerfectScore = grades.some(x => x === 100);

console.log(hasPerfectScore); // true (так, одна сотня є)

// Чи є в масиві від'ємні оцінки?
const hasNegative = grades.some(x => x < 0);

console.log(hasNegative); // false (таких немає)
```

ЗАВДАННЯ ДЛЯ САМОСТІЙНОГО ВИКОНАННЯ

Задача 1. Фільтрація: «Особливий список»

Дано масив рядків (прізвища студентів або назви міст).

Завдання: Створити новий масив, до якого увійдуть лише ті назви, довжина яких більша за 5 символів.

Підказка: Використовуйте властивість `.length` у рядка всередині `filter`.

Задача 2. Трансформація: «Конвертер валют»

Дано масив цін у гривнях: [100, 250, 500, 1200].

Завдання: Створити новий масив, де кожна ціна конвертована в долари (наприклад, поділена на 40).

Метод: `map`.

Задача 3. Агрегація: «Рахунок у грі»

Дано масив набраних очок за 5 раундів: [10, 0, 15, 20, 5].

Завдання: Обчислити загальну суму очок та знайти середній бал.

Метод: `reduce`.

Задача 4. Комбінування: «Математичний фільтр»

Дано масив випадкових чисел: [2, -5, 10, -3, 8, 15, 0].

Завдання:

1. Залишити тільки додатні числа (filter).
2. Кожне з них піднести до квадрата (map).
3. Вивести отриманий масив у консоль.

Задача 5. Пошук: «Цензура»

Дано масив слів. Перевірити:

- Чи є в масиві хоча б одне слово «заборонено» (метод some).
- Чи всі слова в масиві написані з великої літери (метод every).